In [20]:
```
#Janvi Yadav B-4
#EXP5

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_mat
```

In [9]:
```
dataset = pd.read_csv("car_evaluation.csv")

# Load dataset with correct column names
column_names = ["buying", "maint", "doors", "persons", "lug_boot", "safety", "cl

dataset = pd.read_csv("car_evaluation.csv", names=column_names, header=None)

print("Dataset Loaded Successfully")
print(dataset.head())
```

```
Dataset Loaded Successfully
  buying  maint doors persons lug_boot safety  class
0  vhigh  vhigh     2       2    small    low  unacc
1  vhigh  vhigh     2       2    small    med  unacc
2  vhigh  vhigh     2       2    small   high  unacc
3  vhigh  vhigh     2       2      med    low  unacc
4  vhigh  vhigh     2       2      med    med  unacc
```

In [10]:
```
print("Shape of Dataset:", dataset.shape)

print("\nColumn Names:")
print(dataset.columns)

print("\nDataset Information:")
dataset.info()
```

```
Shape of Dataset: (1728, 7)

Column Names:
Index(['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class'], dty
pe='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1728 entries, 0 to 1727
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   buying    1728 non-null   object
 1   maint     1728 non-null   object
 2   doors     1728 non-null   object
 3   persons   1728 non-null   object
 4   lug_boot  1728 non-null   object
 5   safety    1728 non-null   object
 6   class     1728 non-null   object
dtypes: object(7)
memory usage: 94.6+ KB
```

In [11]:
```
print("Missing Values in Dataset:")
```

```
print(dataset.isnull().sum())
```

```
Missing Values in Dataset:
buying      0
maint       0
doors       0
persons     0
lug_boot    0
safety      0
class       0
dtype: int64
```

In [12]:
```
encoder = LabelEncoder()

for column in dataset.columns:
    dataset[column] = encoder.fit_transform(dataset[column])

print("Encoded Dataset:")
dataset.head()
```

```
Encoded Dataset:
```

Out[12]:

	buying	maint	doors	persons	lug_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

In [8]:
```
print(dataset.columns)
```

```
Index(['vhigh', 'vhigh.1', '2', '2.1', 'small', 'low', 'unacc'], dtype='object')
```

In [13]:
```
X = dataset.drop("class", axis=1)
y = dataset["class"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Features Shape: (1728, 6)
Target Shape: (1728,)
```

In [14]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (1382, 6)
Testing Data Shape: (346, 6)
```

In [15]:
```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully")
```

```
Random Forest Model Trained Successfully
```

In [16]:
```
y_pred = model.predict(X_test)

print("Predicted Values:")
print(y_pred[:20])
```

```
Predicted Values:
[2 2 2 0 2 0 2 2 2 2 3 1 2 2 2 2 2 2 2 0]
```

In [17]:
```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy of Random Forest Classifier:", accuracy)
```

```
Accuracy of Random Forest Classifier: 0.9739884393063584
```

In [18]:
```
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[ 75   6   2   0]
 [  0  11   0   0]
 [  0   0 235   0]
 [  1   0   0  16]]
```

In [19]:
```
report = classification_report(y_test, y_pred)

print("Classification Report:")
print(report)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.90      0.94        83
           1       0.65      1.00      0.79        11
           2       0.99      1.00      1.00       235
           3       1.00      0.94      0.97        17

    accuracy                           0.97       346
   macro avg       0.91      0.96      0.92       346
weighted avg       0.98      0.97      0.98       346
```